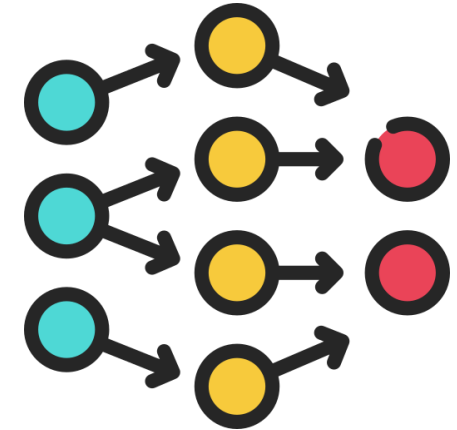




Convolutional Neural Networks (CNN)

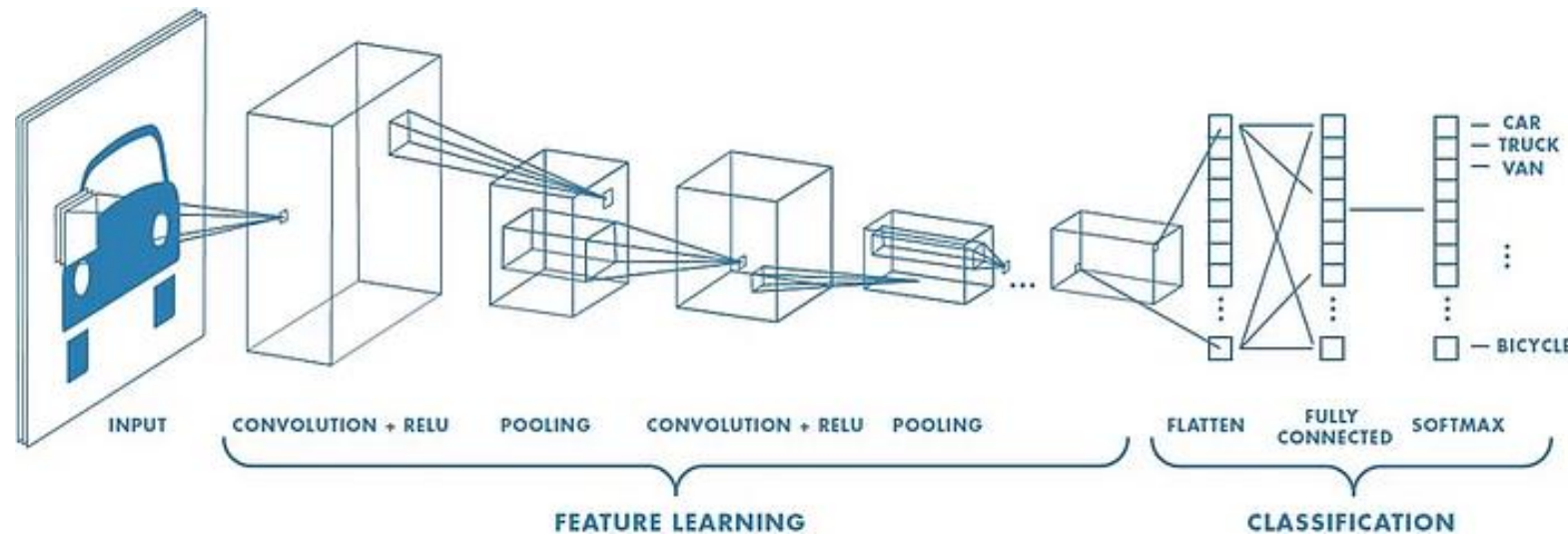


Convolutional Neural Networks (CNN)

A **convolutional neural network** or CNN is a type of **artificial neural network** used in image recognition and processing that is inspired by the biological processes in the **visual cortex** of animals.

CNNs apply filters (small rectangles) to an input image to detect features like edges or shapes

Filters **slide** over the **width** and **height** of the input image and **compute dot products** between the filter and the input to produce an **activation map**.

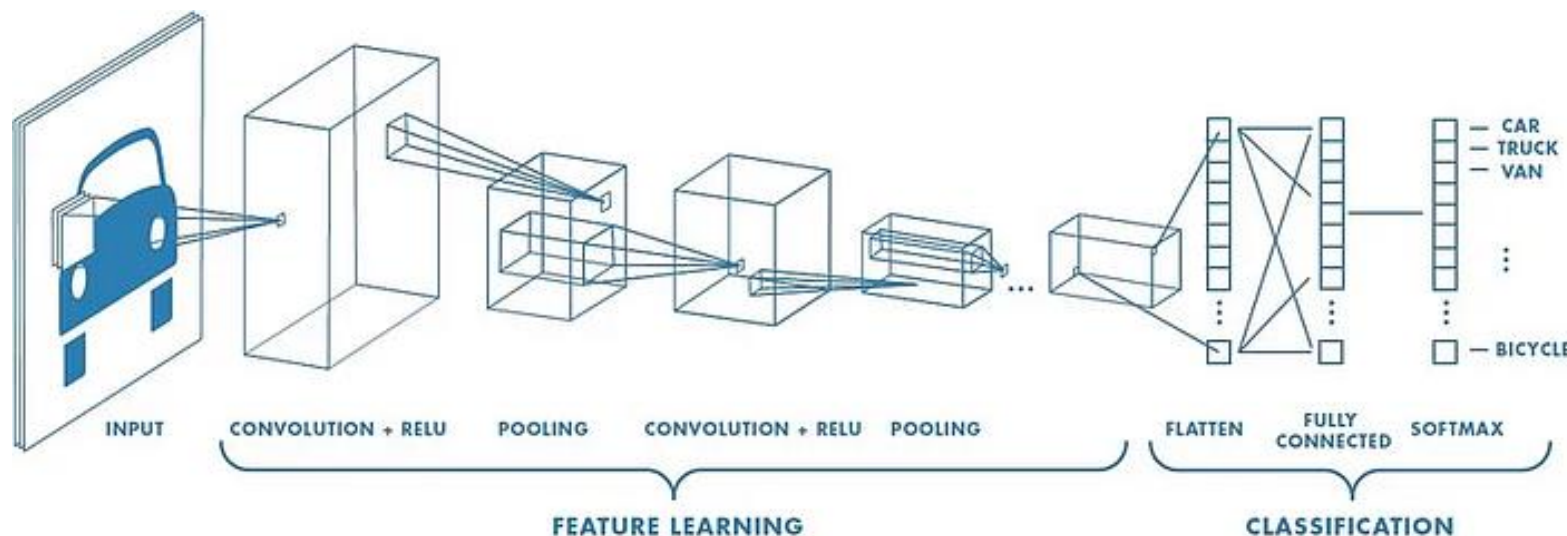


Convolutional Neural Networks (CNN)

Activation maps -> **Pooling layers** that downsample the maps

The final layers are **fully connected** layer.

The last layer is a fully connected layer that produces **classification output**.



Convolutional Layers

Convolutional layers apply a **convolution operation** to the input, passing a **filter** over the entire image

According to Wikipedia Convolution is a mathematical operation which can be performed on two functions to generate third function that shows how shape of one modified by other.

In blue the **original image** tensor

In green the **image tensor** after

0	1	2
2	2	0
0	1	2

3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0



Convolutional Layers: Padding and Stride

PADDING

Convolution (application of Kernel), reduces the size of the output tensor for the image.

Pixel on the corner of the image losses its importance

Padding add some extra pixels to the border of the image

STRIDE

Stride is how the kernel moves across the image

(1,1) -> kernel moves across the width and height by 1 pixel at a time

- a) kernel moves across the width by 1 pixel
- b) kernel moves moves 1 pixel in height and again repeats a

(2,2) the kernel moves across the image tensor by two pixels.

0	0	0	0	0	0
0	35	19	25	6	0
0	13	22	16	53	0
0	4	3	7	10	0
0	9	8	1	3	0
0	0	0	0	0	0

35	19	25	6
13	22	16	53
4	3	7	10
9	8	1	3

35	19	25	6
13	22	16	53
4	3	7	10
9	8	1	3

35	19	25	6
13	22	16	53
4	3	7	10
9	8	1	3

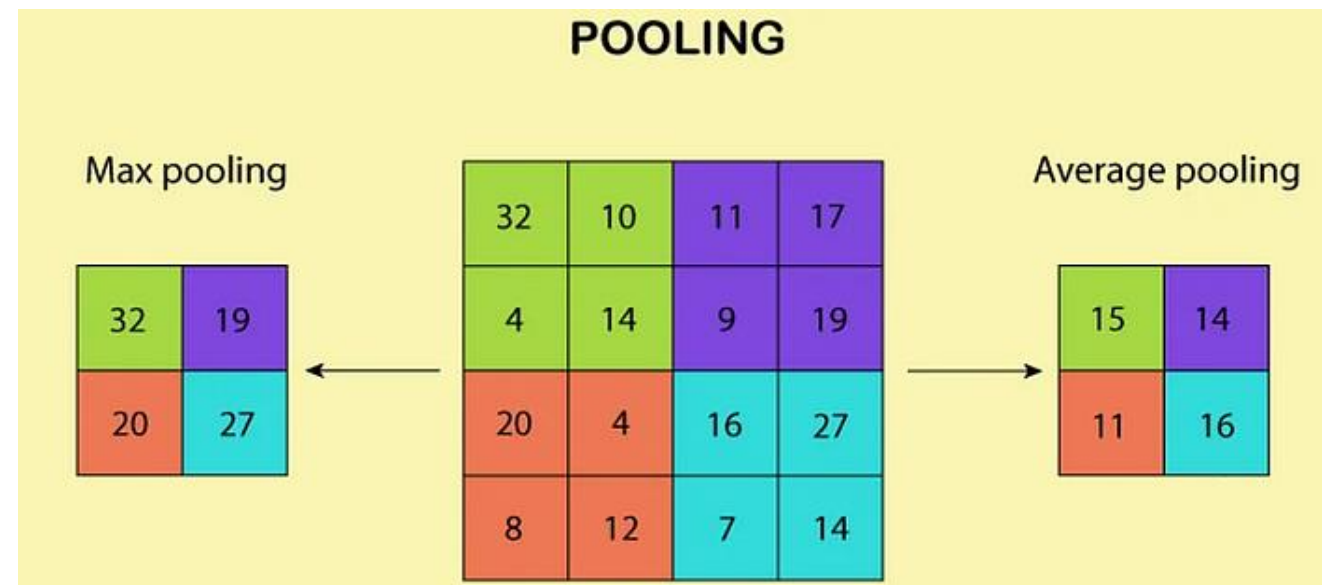
Pooling Layers

Pooling layers are special layer

- inserted **between** convolutional layers.
- downsample the feature maps to **reduce** the number of **parameters**
- control overfitting
- make network **invariant** to **small translations**
- Defined by **filter size** and **stride**

Type of Pooling

- Max Pooling
- Average Pooling
- Sum Pooling

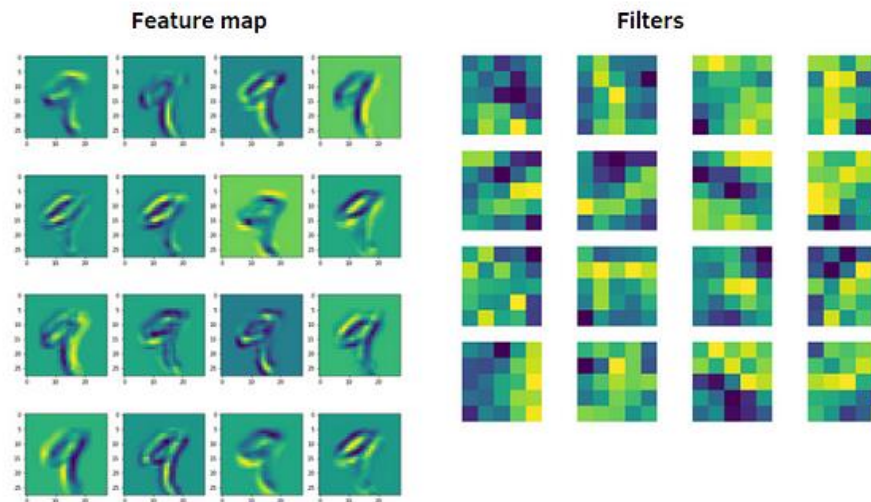


Feature Map

A feature map is the **output** of a **convolutional layer** after applying filters to the input image.

Each **feature map** corresponds to a different **filter** and captures different **features** from the input.

Stacking multiple feature maps together forms a multi-channel output that serves as the **input** for the **next layer**.



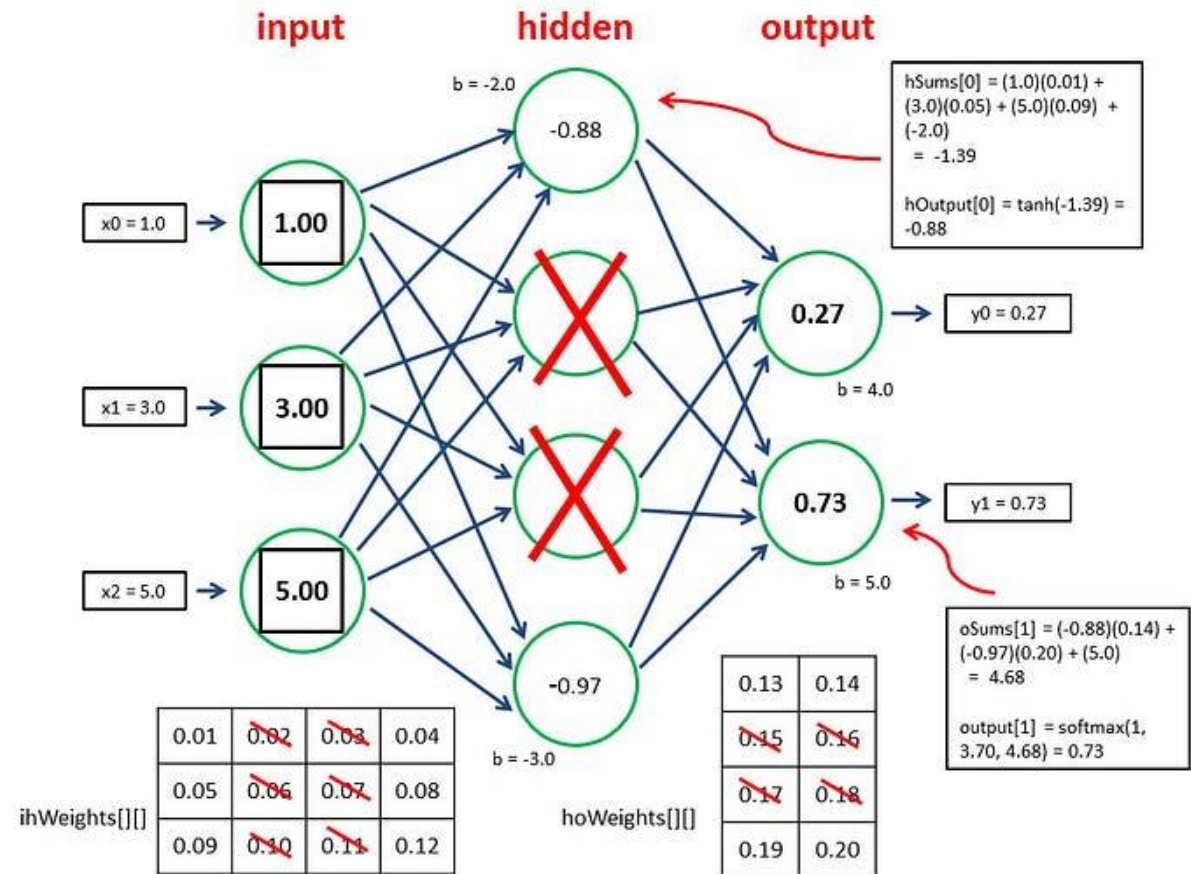
Dropout layer

The **dropout** layer randomly drop-out neurons during **training**.

The model does **not memorize** the training data in that step

This way, the model **generalizes** to new, unseen data.

Used to prevent **overfitting** by



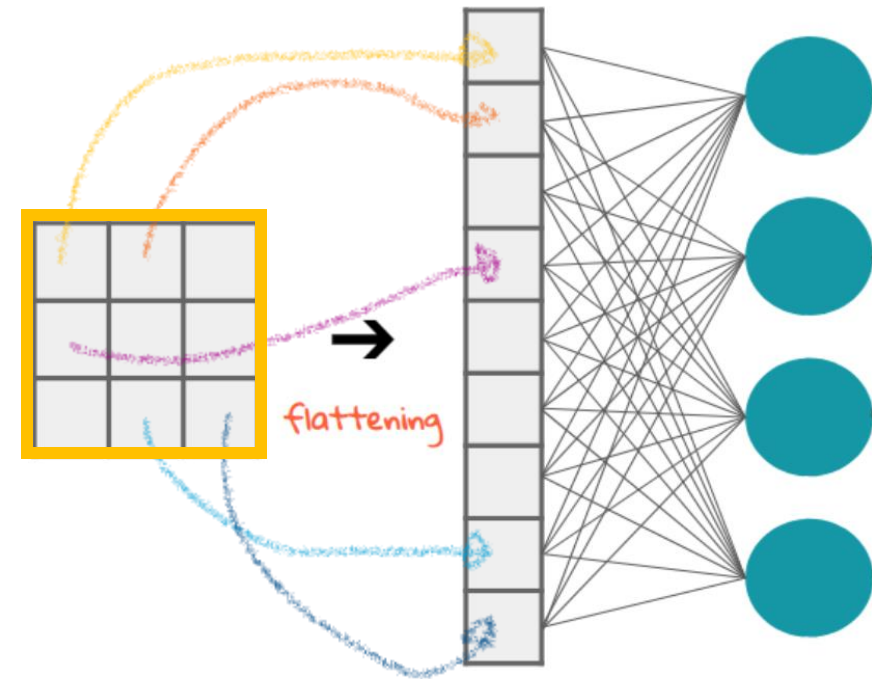
Flattening

Flattening transform **2D tensor** in a **1D tensor**

Flattening maps **Convolutional** to **Fully Connected**

1D tensors are suitable for standard **Neural Network** processing

Several **Fully Connected** layers bring to final final Classification



Benefits of Convolutional Neural Network:

Feature extraction: CNNs are capable of automatically extracting relevant features from an input image, reducing the need for manual feature engineering.

Spatial invariance: CNNs can recognize objects in an image regardless of their location, size, or orientation, making them well-suited to object recognition tasks.

Robust to noise: CNNs can often handle noisy or cluttered images, making them useful for real-world applications where image quality may be variable.

Transfer learning: CNNs can leverage pre-trained models, reducing the amount of data and computational resources required to train a new model.

Performance: CNNs have demonstrated state-of-the-art performance on a range of computer vision tasks, including image classification, object detection, and semantic segmentation.

Limitations of Convolutional Neural Network:

Computational cost: Training a deep CNN can be computationally expensive, requiring significant amounts of data and computational resources.

Overfitting: Deep CNNs are prone to overfitting, especially when trained on small datasets, where the model may memorize the training data rather than generalize to new, unseen data.

Lack of interpretability: CNNs are considered to be a “black box” model, making it difficult to understand why a particular prediction was made.

Limited to grid-like structures: CNNs are limited to grid-like structures and cannot handle irregular shapes or non-grid-like data structures.

PyTorch uses two objects for data managing:

- **Dataset**: stores the samples and their corresponding labels
 - `__len__(self)`
 - `__getitem__(self, idx)`
- **Dataloader**: wraps an iterable around the Dataset to enable easy access to the samples (batch, multiprocessing).

```
class CustomImageDataset(Dataset):  
    def __init__(self, paths, labels, transform=None):  
        self.img_labels = labels  
        self.img_dir = paths  
        self.transform = transform  
  
    def __len__(self):  
        return len(self.img_labels)  
  
    def __getitem__(self, idx):  
        img_path = f"{self.img_dir[idx]}"  
        image = Image.open(img_path).convert("RGB")  
        label = torch.tensor(int(self.img_labels[idx]))  
        if self.transform:  
            image = self.transform(image)  
        return image, label
```

Data Augmentation

In case of data **scarcity**, the application of **random transformation** is a strategy to obtain a **data augmentation**:

Some examples of data augmentation are:

<u>CenterCrop</u> (size)	Crops the given image at the center.
<u>ColorJitter</u> ([brightness, contrast, ...])	Randomly change the brightness, contrast, saturation and hue of an image.
<u>FiveCrop</u> (size)	Crop the given image into four corners and the central crop.
<u>Grayscale</u> ([num_output_channels])	Convert image to grayscale.
<u>Pad</u> (padding[, fill, padding_mode])	Pad the given image on all sides with the given "pad" value.
<u>RandomAffine</u> (degrees[, translate, scale, ...])	Random affine transformation of the image keeping center invariant.
<u>RandomApply</u> (transforms[, p])	Apply randomly a list of transformations with a given probability.
<u>RandomCrop</u> (size[, padding, pad_if_needed, ...])	Crop the given image at a random location.
<u>RandomGrayscale</u> ([p])	Randomly convert image to grayscale with a probability of p (default 0.1).
<u>RandomHorizontalFlip</u> ([p])	Horizontally flip the given image randomly with a given probability.
<u>RandomPerspective</u> ([distortion_scale, p, ...])	Performs a random perspective transformation of the given image with a given probability.

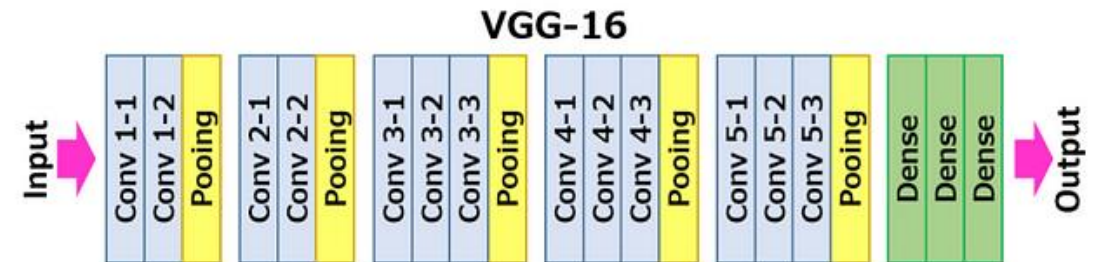
<u>RandomResizedCrop</u> (size[, scale, ratio, ...])	Crop a random portion of image and resize it to a given size.
<u>RandomRotation</u> (degrees[, interpolation, ...])	Rotate the image by angle.
<u>RandomVerticalFlip</u> ([p])	Vertically flip the given image randomly with a given probability.
<u>Resize</u> (size[, interpolation, max_size, ...])	Resize the input image to the given size.
<u>TenCrop</u> (size[, vertical_flip])	Crop the given image into four corners and the central crop plus the flipped version of these (horizontal flipping is used by default).
<u>GaussianBlur</u> (kernel_size[, sigma])	Blurs image with randomly chosen Gaussian blur.
<u>RandomInvert</u> ([p])	Inverts the colors of the given image randomly with a given probability.
<u>RandomPosterize</u> (bits[, p])	Posterize the image randomly with a given probability by reducing the number of bits for each color channel.
<u>RandomSolarize</u> (threshold[, p])	Solarize the image randomly with a given probability by inverting all pixel values above a threshold.
<u>RandomAdjustSharpness</u> (sharpness_factor[, p])	Adjust the sharpness of the image randomly with a given probability.
<u>RandomAutocontrast</u> ([p])	Autocontrast the pixels of the given image randomly with a given probability.
<u>RandomEqualize</u> ([p])	Equalize the histogram of the given image randomly with a given probability.

VGG 16

VGG-16 is a convolutional neural network (CNN) model with 16 layers (13 convolutional layers and 3 fully connected layers), known for its depth.

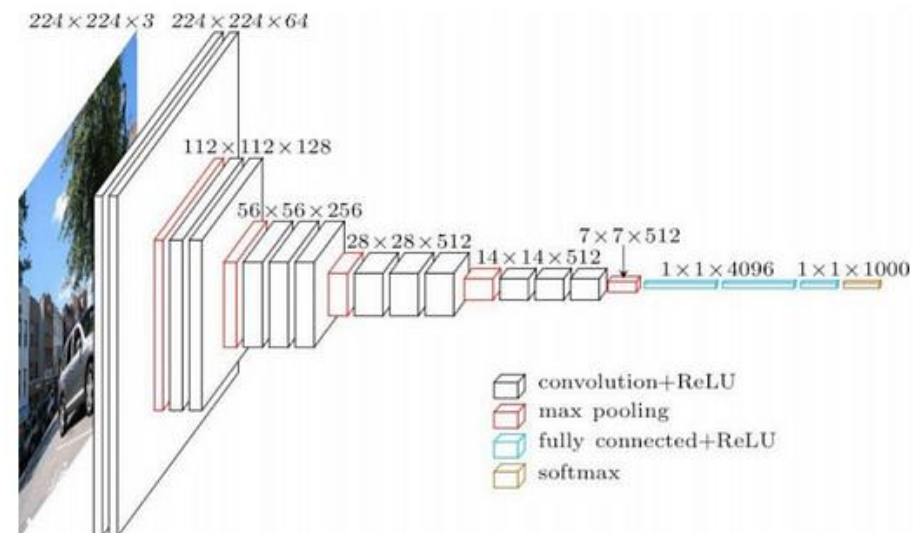
A convolutional neural network model proposed by [K. Simonyan](#) and [A. Zisserman](#) from the University of Oxford in the paper "*Very Deep Convolutional Networks for the Large-Scale Image Recognition*"

The model achieves 92.7% top 5 test accuracy in ImageNet



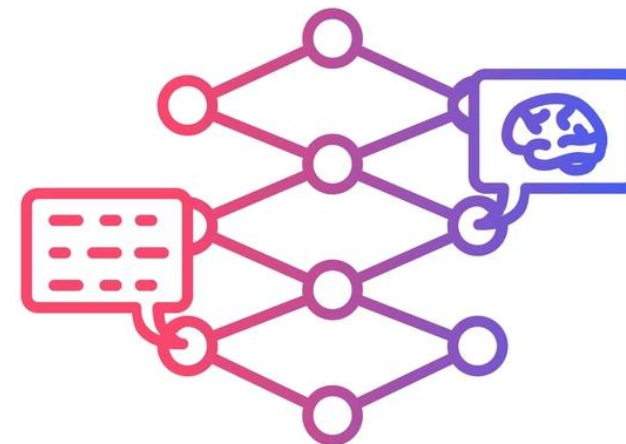
The Architecture

The architecture depicted below is VGG16.





Autoencoder



Autoencoder

Autoencoders are a type of unsupervised learning technique.

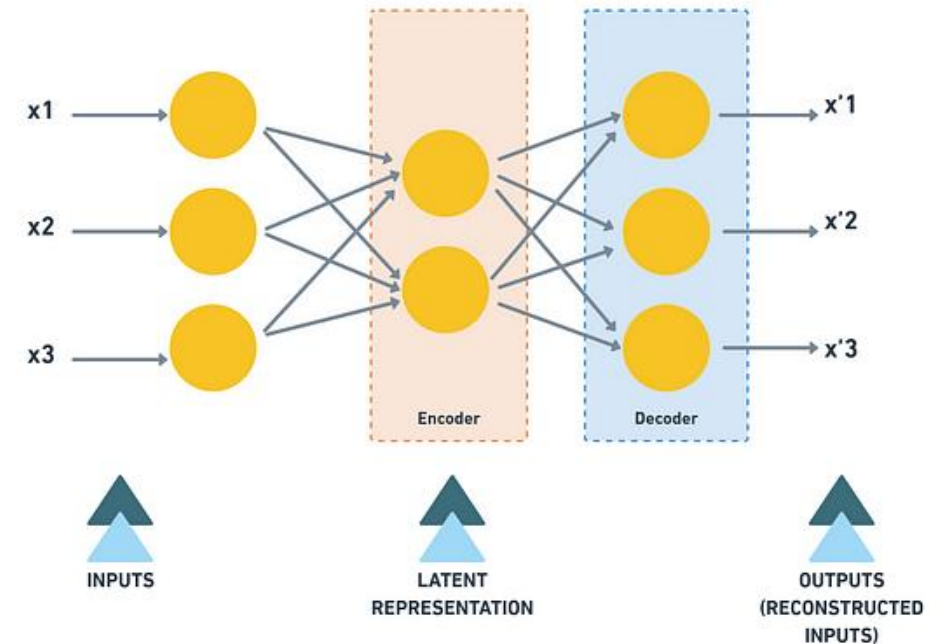
They are capable of finding **hidden features** or information in raw input data using artificial neural networks.

In a nutshell it is a type of neural network that finds the function **mapping** the **features** x to **itself**.

*An autoencoder is a neural network that is trained to reconstruct its input. It consists of two parts: an **encoder** and a **decoder**.*

The **encoder** takes the input and compresses it into a lower-dimensional representation

The **decoder** takes this compressed representation and reconstructs the original input.



Autoencoder

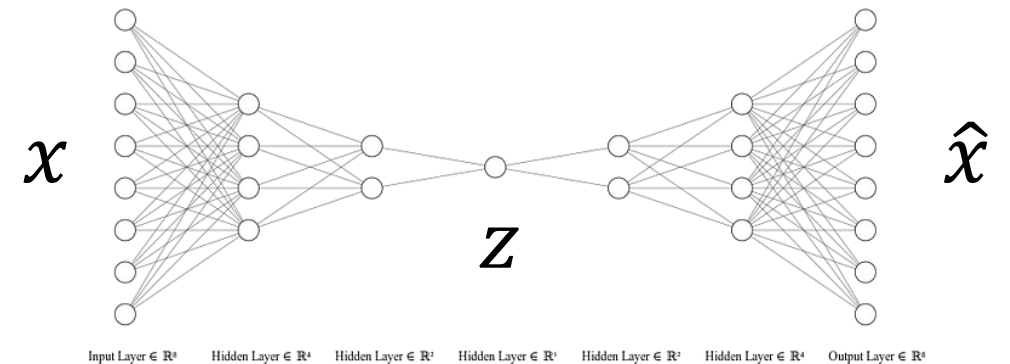
An autoencoder is a type of neural network that finds the function **mapping** the **features x** to **itself**.

Autoencoder needs a loss function that measures how well the decoder can **reconstruct** the input from the **hidden representation**

$$z = h_e(x)$$

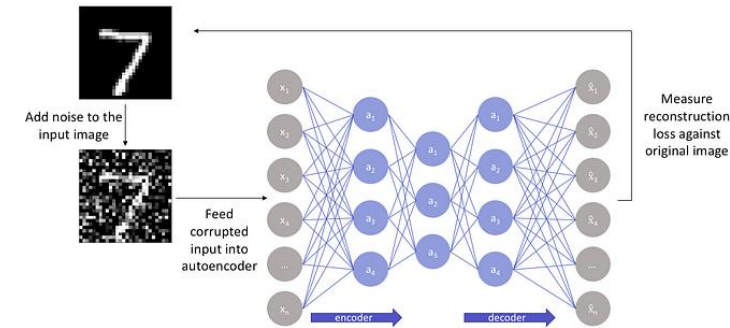
$$\hat{x} = h_d(z)$$

$$MSE = \left(\frac{1}{N}\right) * \sum (x - \hat{x})^2$$



Why use Autoencoder

- Dimensionality Reduction (like PCA, but nonlinear)
- Denoising corrupted data
- Anomaly Detection
- Pretraining / Feature extraction
- Image Compression and Generation



Convolutional Autoencoder

A **Convolutional Autoencoder** (CAE) is a type of autoencoder designed specifically for image data

The **encoder layers** are typically referred to as **convolutional layers** because they perform convolution operations on the input image to extract **features**

The **decoder layers** are often called **deconvolution layers** because they use deconvolution or transpose convolution operations to reconstruct the **output image**

The lower-dimensional latent space is reshaped into a 3D tensor that can be processed by the **upsampling** layers of the decoder

